

Machine à balles

Madina a inventé une machine à balles pour divertir les participants des IOI. La structure interne de la machine est la suivante :

- La machine est composée de N **nœuds**, indexés de 0 à $N - 1$. Le nœud $N - 1$ est appelé la **racine**.
- Pour chaque nœud u tel que $0 \leq u < N - 1$, le **parent** du nœud u est le nœud $P[u]$, qui vérifie $P[u] > u$. Le nœud u est un **enfant** de $P[u]$. La racine n'a pas de parent. Un nœud sans enfant est appelé une **feuille**.

Vous ne connaissez ni N , ni le parent $P[u]$ d'aucun nœud u . Madina vous indique en revanche M , le nombre de feuilles, et que les indices des feuilles sont $0, 1, \dots, M - 1$. Votre tâche consiste à déterminer la structure interne de la machine en l'utilisant.

Chaque nœud de la machine peut contenir au maximum une **balle**, et chaque balle a une **valeur** qui est un entier positif ou nul. On dit qu'un nœud a pour valeur x s'il contient une balle de valeur x . Un nœud est **occupé** s'il contient une balle ; sinon, il est **libre**. Initialement, tous les nœuds sont libres.

Vous pouvez effectuer deux types d'opérations :

- **insert** (U, X) : essayer d'insérer une nouvelle balle de valeur X dans la feuille U .
 - Si la feuille U est occupée, l'opération renvoie `false` sans insérer la balle.
 - Si la feuille U est libre, la balle est placée au nœud U . Ensuite, elle se déplace de manière répétée vers le parent de son nœud courant tant que ce parent est libre. Le déplacement s'arrête lorsque la balle atteint la racine ou un nœud dont le parent est occupé. L'opération renvoie `true`.
- **collect()** : si la racine est libre (c'est-à-dire si la machine est vide), `collect` renvoie un tableau vide. Sinon, la procédure récursive `traverse` décrite ci-dessous écrit les valeurs de toutes les boules présentes dans la machine dans un tableau S . Initialement, le tableau S est vide et `traverse(N - 1)` est appelé.

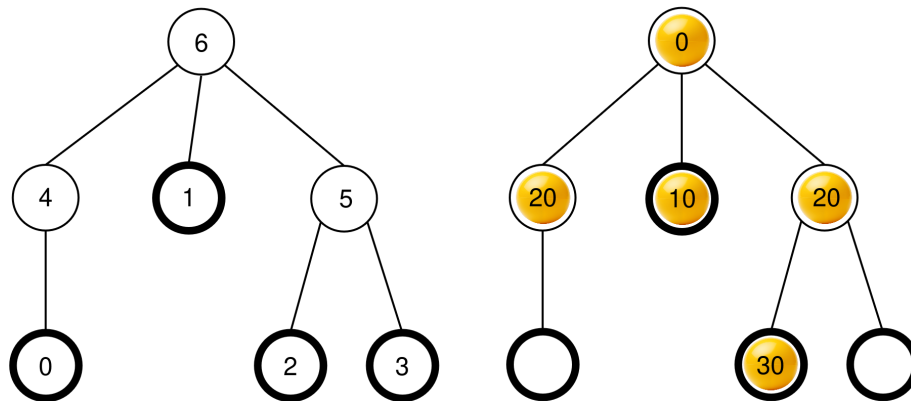
```

traverse(u):
    insérer la valeur de la balle du nœud u à la fin de S
    soit c[u] la liste des enfants occupés de u
    trier c[u] par valeur de balle associée croissante
        (si plusieurs sommets ont la même valeur,
         ils peuvent apparaître dans n'importe quel ordre)
    pour chaque v dans c[u]:
        traverse(v)

```

Une fois cette procédure terminée, **toutes les balles sont retirées** de la machine, et `collect` renvoie S .

Par exemple, considérons une machine avec $N = 7$ nœuds et un tableau de parents $P = [4, 6, 5, 5, 6, 6]$. L'image de gauche affiche la machine vide avec les indices des nœuds, tandis que l'image de droite montre une configuration possible de boules après une séquence d'opérations `insert` (cette séquence est donnée dans la section Exemple).



Lorsque `collect()` est appelé, la racine (nœud 6) est occupée, donc S est initialisé avec $S = []$ et `traverse(6)` est appelé.

traverse(6) : le nœud 6 a pour valeur 0 ; l'ajouter à S donne $S = [0]$. Les enfants du nœud 6 sont 4, 1 et 5, tous occupés. Leurs valeurs sont respectivement 20, 10 et 20 . Un tri par ordre croissant donne $c[6] = [1, 5, 4]$ (notez que $c[6] = [1, 4, 5]$ serait également valide, puisque les nœuds 4 et 5 ont la même valeur). La procédure appelle ensuite `traverse` sur chaque nœud de $c[6]$ dans cet ordre.

- **traverse(1)** : le nœud 1 a pour valeur 10 ; l'ajouter à S donne $S = [0, 10]$. Le nœud 1 n'a pas d'enfants occupés, donc cet appel se termine.
- **traverse(5)** : le nœud 5 a pour valeur 20 ; l'ajouter à S donne $S = [0, 10, 20]$. Le nœud 5 a un enfant occupé, le nœud 2 , donc $c[5] = [2]$ et la procédure appelle `traverse(2)` .
 - **traverse(2)** : le nœud 2 a pour valeur 30 ; l'ajouter à S donne $S = [0, 10, 20, 30]$. Le nœud 2 n'a pas d'enfants occupés, donc cet appel se termine.

- L'exécution retourne à `traverse(5)` qui a traité tous les enfants dans $c[5]$, son appel se termine donc.
- **`traverse(4)`** : le nœud 4 a pour valeur 20 ; l'ajouter à S donne $S = [0, 10, 20, 30, 20]$. Le nœud 4 n'a pas d'enfants occupés, donc cet appel se termine.

Tous les appels sont terminés et il n'y a plus de nœuds à traiter. Enfin, chaque balle est retirée de la machine et `collect` renvoie le tableau $S = [0, 10, 20, 30, 20]$.

Votre tâche consiste à déterminer le nombre de nœuds N et un tableau de parents $R = [R[0], R[1], \dots, R[N-2]]$ décrivant la structure de la machine, avec possiblement une étiquette différente pour les nœuds qui ne sont ni des feuilles ni la racine. Formellement, un tableau R fourni est considéré comme correct s'il est possible d'attribuer à chaque nœud u de la machine une étiquette distincte $L[u]$ avec $0 \leq L[u] < N$ et

- $L[u] = u$ pour tout $0 \leq u < M$ et $u = N-1$, et
- $L[P[u]] = R[L[u]]$ pour tout $0 \leq u < N-1$.

Il n'est **pas** nécessaire que $R[u] > u$. La machine **doit être vide** lorsque vous renvoyez votre solution.

Soit K le nombre d'opérations `collect` effectuées, et B la valeur maximale d'une boule lors des appels à `insert`. Soit $C = K + B$. Alors C **ne doit pas dépasser** 1000. Votre score dans certaines des sous-tâches dépend de la valeur de C .

Détails d'implémentation

Vous devez implémenter la fonction suivante.

```
std::vector<int> find_structure(int M)
```

- M : le nombre de feuilles dans la machine.
- La procédure est appelée exactement une fois pour chaque fichier test.
- La procédure doit renvoyer un tableau $R = [R[0], R[1], \dots, R[N-2]]$ de longueur $N-1$, un tableau correct de parents.

Pour interagir avec la machine, votre procédure peut appeler les deux procédures suivantes.

La première procédure est :

```
bool insert(int U, int X)
```

- U : l'indice de la feuille où la bille doit être insérée. La condition $0 \leq U < M$ doit être respectée.

- X : la valeur entière de la balle. La condition $0 \leq X \leq 1000$ doit être respectée.
- Cette procédure renvoie `true` si la balle a été insérée avec succès, ou `false` si la feuille U était déjà occupée.
- Cette procédure peut être appelée au maximum 500 000 fois.

La deuxième procédure est :

```
std::vector<int> collect()
```

- Collecte les valeurs de toutes les balles insérées, vide la machine et renvoie le tableau collecté S .

Si à un moment quelconque de l'exécution de votre programme la valeur de C dépasse 1000, votre solution recevra le verdict `Output isn't correct: Too many resources used`.

La structure de la machine est fixée avant l'appel à `find_structure`. L'évaluateur (grader) est **déterministe** : si vous le lancez deux fois avec la même séquence d'opérations, les appels à `collect` vont renvoyer les mêmes tableaux.

Contraintes

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Sous-tâches

Sous-tâche	Score	Contraintes supplémentaires
1	5	La racine a exactement M enfants.
2	10	$M \leq 3$
3	25	$N \leq 200$, $M \leq 45$
4	60	Aucune contrainte supplémentaire.

Dans les sous-tâches 3 et 4, votre score dépend de la valeur de C comme suit.

Sous-tâche 3 (25 points)

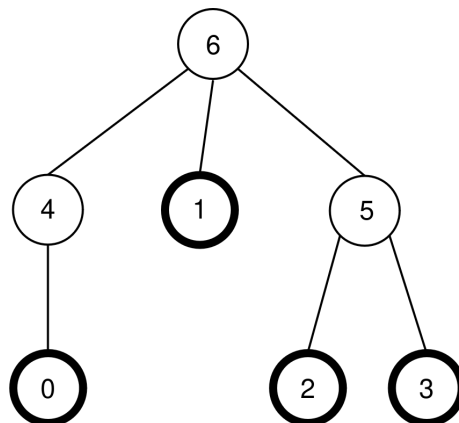
Limites	Score
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Sous-tâche 4 (60 points)

Limites	Score
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Exemple

Considérons la même machine que dans la description de la tâche, donc $N = 7$, $M = 4$ et $P = [4, 6, 5, 5, 6, 6]$. La machine est à nouveau représentée dans la figure suivante :



L'évaluateur (grader) effectue l'appel suivant :

```
find_structure(4)
```

La procédure peut effectuer la séquence d'appels suivante :

1. **insert(0, 0)** : la feuille 0 est libre, donc une balle de valeur 0 est placée sur la feuille 0 . Le nœud $P[0] = 4$ est libre, donc la balle se déplace vers le nœud 4. Le nœud $P[4] = 6$ est libre,

donc la balle se déplace vers le nœud 6. Comme le nœud 6 est la racine, le mouvement s'arrête. L'appel renvoie `true`.

2. `insert(3, 20)` : la feuille 3 est libre, donc une balle de valeur 20 est placée sur la feuille 3. Le nœud $P[3] = 5$ est libre, donc la balle se déplace vers le nœud 5. Comme $P[5] = 6$ est occupé, le déplacement s'arrête. L'appel renvoie `true`.
3. `insert(1, 10)` : la feuille 1 est libre, donc une balle de valeur 10 est placée sur la feuille 1. Comme $P[1] = 6$ est occupé, la balle reste sur le nœud 1. L'appel renvoie `true`.
4. `insert(2, 30)` : la feuille 2 est libre, donc une balle de valeur 30 est placée sur la feuille 2. Comme $P[2] = 5$ est occupé, la balle reste sur le nœud 2. L'appel renvoie `true`.
5. `insert(1, 25)` : la feuille 1 est occupée, la balle n'est donc pas placée sur la feuille 1. L'appel renvoie `false`.
6. `insert(0, 20)` : la feuille 0 est libre, donc une balle de valeur 20 est placée sur la feuille 0. Le nœud $P[0] = 4$ est libre, donc la balle se déplace vers le nœud 4. Comme $P[4] = 6$ est occupé, le déplacement s'arrête. L'appel renvoie `true`.

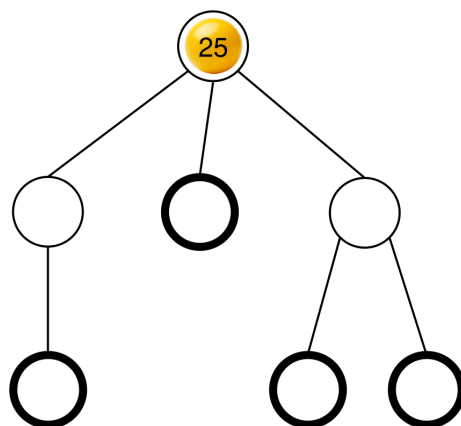
La configuration de balles résultante a déjà été présentée plus haut.

Ensuite, la procédure peut appeler :

```
collect()
```

L'exécution de cet appel a déjà été expliquée dans la description de la tâche ; cet appel renvoie donc $S = [0, 10, 20, 30, 20]$. Ensuite, la machine est à nouveau vide.

Ensuite, la procédure peut appeler `insert(2, 25)`. La feuille 2 étant libre, une bille de valeur 25 est placée sur la feuille 2. Cette bille se déplace ensuite vers le nœud 5, puis vers le nœud 6, où elle s'arrête. L'appel renvoie `true`. La configuration résultante des billes est illustrée dans la figure suivante.



Enfin, la procédure peut appeler `collect()` qui renvoie $S = [25]$ et vide la machine.

Il existe deux tableaux parents corrects que `find_structure` peut renvoyer : $R = P = [4, 6, 5, 5, 6, 6]$ (qui correspond à l'étiquetage $L = [0, 1, 2, 3, 4, 5, 6]$) et $R = [5, 6, 4, 4, 6, 6]$

(qui correspond à l'étiquetage $L = [0, 1, 2, 3, 5, 4, 6]$). Dans cet exemple, $K = 2$ et $B = 30$, donc $C = 32$.

Évaluateur d'exemple (grader)

Format d'entrée :

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Format de sortie :

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Ici, Q est la longueur du tableau R renvoyé par `find_structure`.